

EUR/USD Price-Direction Predictor — Design Documentation

Companion document to the code in `(src/)`. This explains **what** each design decision was, **why** it was made, what the results actually mean, and what would have to change to move from a coursework model to a live system.

1. Pipeline overview

```
src/fetch_eurusd.py      → data/raw/eurusd_ohlcv.csv      (raw OHLCV)
src/indicators.py        → (shared library, not run directly)
src/build_features.py    → data/processed/eurusd_features.csv (features + labels)
src/train_model.py       → models/*.joblib, outputs/training_metrics.json
src/plot_chart.py        → outputs/eurusd_chart_*.png
```

`(indicators.py)` is imported by **both** `(build_features.py)` and `(plot_chart.py)`. That is deliberate: the indicator values the model trains on are computed by the exact same code that draws the chart. If those were two separate implementations, the chart could show one thing while the model learned another — a bug class that is very hard to notice.

Run order:

```
bash

cd src
python3 fetch_eurusd.py      # only when refreshing data
python3 build_features.py
python3 train_model.py
python3 plot_chart.py
```

2. Dataset

Property	Value
Instrument	EUR/USD
Source	Dukascopy (free, no API key, bank-grade tick archive)
Granularity	1 hour, bid side
Range	2026-01-05 → 2026-09-04
Raw rows	4,177
Rows after indicator warm-up + label shift	4,156

Why hourly, not 1-minute or daily. Daily bars over 9 months give ~190 rows — nowhere near enough to train on. 1-minute bars give ~250k rows but are dominated by microstructure noise and spread effects that a feature set like this cannot model. Hourly is the middle ground: enough samples to train, coarse enough that the signal-to-noise ratio is not hopeless.

Why EUR/USD rather than an Indian equity. It trades ~24h on weekdays, is the most liquid instrument in the world, and has free deep history with no broker account required. Indian equity data at this granularity is either paid or scraped from unofficial endpoints (see §7).

3. Indicator design — the registry pattern

Each indicator in `indicators.py` is a plain function taking the OHLCV frame and returning a `Series` (single column) or `DataFrame` (multi-column). It is then listed in `INDICATOR_REGISTRY`.

```
python
```

```
def my_signal(df: pd.DataFrame) -> pd.Series:
    ...
    return series.rename("my_signal")

INDICATOR_REGISTRY["my_signal"] = my_signal
```

Why a registry rather than hard-coded calls. With 7 people on the project, adding an indicator should not require editing the feature builder, the chart script, and the trainer. Register once, and `apply_indicators()` picks it up everywhere. This is the single most important structural decision in the codebase for team workflow.

3.1 The ATR trailing stop (Pine Script conversion)

The supplied Pine Script ("Krishna's ATR Bot") was converted to `atr_trailing_stop()` with defaults preserved exactly: `key_value=3.0`, `atr_period=10`.

Why this one needs a loop when the others do not. RSI, MACD, and Bollinger Bands are *vectorisable* — every output value depends only on a fixed window of inputs, so pandas `rolling()`/`ewm()` computes the whole column at once. The ATR trailing stop is **path-dependent**: bar n 's stop depends on bar $n-1$'s stop, which depends on $n-2$, and so on back to the start. That is exactly what Pine's `var float xATRTrailingStop = na` expresses — state persisting across bars. There is no rolling-window equivalent, so the Python version uses an explicit `for` loop over bars, mirroring how Pine executes.

What was deliberately dropped. The original script's `label.new(...)` error overlays and the ATR(14) fallback are chart annotations and debugging aids. They change what a human sees on a TradingView chart; they do not change any computed value. Reproducing them in a feature pipeline would add code with no effect on the numbers. The signal mathematics is preserved unchanged.

Outputs: `atr_trailing_stop` (the level), `atr_position` (1/-1/0), `atr_buy_signal`, `atr_sell_signal`.

4. Modelling decisions

4.1 Chronological split — never random

```
python
```

```
train, val, test = df[:70%], df[70:85%], df[85:100%]
```

This is the single most important correctness decision in `train_model.py`. `train_test_split(shuffle=True)` on a time series lets the model see bars from *after* the test period during training. Accuracy shoots up, the model looks excellent, and it is entirely an artefact. Examiners probe for this.

4.2 Walk-forward validation

A single train/val split tests the model in exactly one market regime. If that regime happens to suit the model, the number flatters it. `walk_forward_validation()` uses `TimeSeriesSplit` to train on an expanding window and test on the block immediately after, five times over. Reporting the **mean and spread** across folds is far more honest, and in this project the two methods disagree — see §5.

4.3 Dropped features — leakage and non-stationarity

`LEAKY_OR_NONSTATIONARY` removes `open/high/low/close`, `sma_*`, `bb_upper/ lower/mid`, and `atr_trailing_stop` from the feature set.

Why. These are all *price levels*. EUR/USD traded around 1.17 in January and 1.16 in September. A tree model splits on thresholds like `close > 1.1683`, which encodes **when** a bar occurred, not what the market was doing. The model then memorises the training period's price range and has nothing useful to say about a range it has never seen. This is the classic non-stationarity trap in financial ML.

What remains is scale-free: returns, RSI, MACD histogram, `bb_percent_b` (position *within* the bands, 0-1), ATR, volatility, `session_range_ratio`, `momentum_zscore`, `atr_position`. These mean the same thing at 1.16 as at 1.30.

Note that `atr_position` (the ± 1 state) is kept while `atr_trailing_stop` (the price level) is dropped — same indicator, but only one of the two is stationary.

4.4 XGBoost as the baseline, not a neural network

- Works directly on tabular indicator rows with no sequence-window plumbing
- Trains in seconds, so the team can iterate on features rather than wait
- Exposes feature importances, which is what makes §5 interpretable
- Establishes a number a neural network must **beat** to justify its cost

Starting with an LSTM would mean not knowing whether any result came from the architecture or from the features.

5. Results — and what they honestly mean

From `outputs/training_metrics.json`:

Split	Accuracy	Baseline	Edge	ROC-AUC
Train	0.710	0.521	+0.189	0.786
Validation	0.535	0.510	+0.024	0.522
Test (held out)	0.548	0.527	+0.021	0.553

Walk-forward across 5 folds: mean accuracy **0.505**, std **0.026**, mean edge **−0.018**.

Read this carefully

The headline test edge of +2.1% is not real skill. Three reasons:

1. **Walk-forward disagrees with the single split.** Mean edge across folds is *negative* (−1.8%), and per-fold edges swing from −7.0% to +2.7%. The positive test number is one draw from a distribution centred near zero.
2. **The fold spread exceeds the effect.** A $\pm 2.6\%$ standard deviation around a +2.1% claimed edge means the result is inside the noise band.
3. **Train/test gap of 0.161** (0.710 vs 0.548) with train ROC-AUC 0.786 vs test 0.553 — the model is still fitting training noise despite `max_depth=3` and regularisation.

`train_model.py` prints this verdict automatically rather than leaving a number near 50% to be quoted as a success.

Why accuracy sits near 50%

This is the expected result, not a coding failure, and it is worth stating plainly in the report:

- EUR/USD is the most liquid market in existence. Any simple, reliably exploitable pattern in hourly bars is arbitrated away within minutes.
- Next-bar direction from technical indicators alone is close to a fair coin. Published academic results on this exact task cluster at 50–55%.
- Roughly 4,100 hourly bars across ~8 months spans only a handful of market regimes — not enough to learn regime-conditional behaviour.
- The label is the hardest possible version of the question. Next-bar sign ignores magnitude, so a 0.001-pip move counts the same as a large one.

Feature importance (top 5): `atr_position` (0.087), `log_return_1` (0.078), `macd_hist` (0.073), `atr` (0.067), `bb_percent_b` (0.065). The supplied ATR indicator ranks first — a genuine result worth reporting, though importance measures which features the model *split on*, not that those splits generalised.

6. Where to go next

6.1 Change the target before changing the model

The highest-leverage change is **not** a fancier architecture. Options, roughly in order of expected payoff:

Change	Rationale
Triple-barrier labelling	Label by which of {take-profit, stop-loss, time limit} is hit first, instead of next-bar sign. Standard in the literature (López de Prado) and produces a far more learnable target.
Multi-bar horizon	Predict 4h or 24h ahead. Longer horizons have better signal-to-noise than next-bar.
Volatility-scaled labels	Only label a move as up/down if it exceeds $k \times ATR$, with a neutral class otherwise. Stops the model chasing sub-spread noise.
Regression on returns	<code>XGBRegressor</code> on <code>target_return</code> , evaluated with directional accuracy on high-confidence predictions only.
More history	Pull 5–10 years rather than 9 months. Dukascopy has it free; this alone may matter more than anything else on this list.

6.2 If moving to a neural network

Only worth attempting **after** the target is fixed and XGBoost has a stable baseline to beat. Realistic ordering:

LSTM / GRU. The natural next step. Feed sequences of shape `(batch, lookback, n_features)` — e.g. 48 hourly bars of the same stationary features — rather than flat rows. What changes structurally:

- Add a `build_sequences()` step producing sliding windows. The chronological split must happen **before** windowing, or windows straddle the boundary and leak.
- Scale features (`StandardScaler`) fitted on **train only**, then applied to val/test. Trees do not need this; neural nets do, and fitting the scaler on the full dataset is a subtle and common leak.
- Expect worse results than XGBoost initially. Sequence models need far more data; ~4k samples is small for an LSTM.

Temporal Convolutional Network (TCN). Often outperforms LSTMs on financial series, trains faster, and handles long lookbacks via dilated convolutions. Worth trying before a Transformer.

Transformer. Only with substantially more data (years of minute bars) and a genuine reason to expect long-range dependencies. On 4k samples it will overfit badly. Do not reach for this because it is fashionable — a deck claiming a

Transformer beat XGBoost on 4,000 rows invites exactly the question you do not want at a viva.

Hybrid (realistic sweet spot). Keep XGBoost on engineered indicator features, and add a small LSTM branch on raw return sequences, then ensemble. Captures both hand-designed structure and learned temporal patterns.

6.3 Evaluation must change too

Accuracy is the wrong end metric for a trading system. Before claiming anything works, add:

- **Directional accuracy on confident predictions only** (e.g. `proba > 0.6`)
- **A backtest with transaction costs** — EUR/USD spread is ~0.5–1 pip; a strategy trading hourly needs a real edge just to break even
- **Sharpe ratio and maximum drawdown**, not classification metrics
- **Purged/embargoed CV** — drop samples adjacent to the split boundary, since overlapping label windows leak across it

A model at 52% accuracy that trades only its top-decile confidence signals can be viable; a model at 56% accuracy that trades every bar can lose money. Report both.

7. Going live, and Indian-market data sources

7.1 What changes for a live system

The batch pipeline here is deliberately file-based. Live operation needs:

1. **Streaming ingestion** — replace `fetch_eurusd.py`'s bulk pull with a scheduled poll or WebSocket subscription writing into the feature store.
2. **Online feature computation** — `apply_indicators()` currently recomputes the whole history. At production rates that is wasteful; the stateful ATR loop in particular should be refactored to update incrementally from the last stored state rather than replaying from bar zero.
3. **Point-in-time correctness** — a feature must only ever use data available at that timestamp. Easy to violate accidentally when bars arrive late or get revised.
4. **Model refresh cadence** — markets drift. Decide on a retraining schedule and monitor for degradation rather than deploying once.

5. **The security layer** — this is where the project's cyber half attaches: API gateway auth on the ingestion path, audit logging of every prediction, and the per-source anomaly detectors cross-validating that an incoming feed has not been tampered with. A price feed is an untrusted input.

7.2 Indian market APIs

Two distinct needs — **historical data for training** and **live data for serving** — and they are often priced separately.

Free / low-cost broker APIs (all require a KYC'd broker account):

Broker	Cost	Notes
Angel One SmartAPI	Free	Designed to make algorithmic trading accessible to retail users with a free API ecosystem, covering automated orders, real-time streaming, and historical data. Official <code>smartapi-python</code> SDK. Commonly recommended as the free baseline.
Fyers API	Free	Free trading API with strong SDK support; users cite free minute-level historical data for 1-2 years — the most relevant point for model training.
Shoonya (Finvasia)	Free	Supports orders, portfolios, positions, live data, depth, and options info across NSE, BSE, and MCX.
DhanHQ	Trading free, data paid	Trading APIs at ₹0 for order placement, data APIs at ₹499 for real-time and historical. Publishes its rate limits clearly.
Zerodha Kite Connect	Paid	Kite Connect Personal gives API access minus market data, so you can place orders if you bring your own data source; commonly quoted around ₹2,000/month. Best documentation and largest community.
Upstox	Per-order	₹10 per executed order via API, extended until 31 March 2026.
Groww	Paid	Rejected earlier: monthly fee plus a funded trading account.

Recommendation for this project: Fyers or Angel One SmartAPI — free, official Python SDKs, and Fyers' free minute-level history is directly useful for training. Avoid paid tiers; the project does not need order execution.

Non-broker options for prototyping:

- `yfinance` with `.NS`/`.BO` suffixes (e.g. `RELIANCE.NS`) — no account, but unofficial and rate-limited
- `jugaad-data` / `nsepython` — unofficial NSE scrapers, more community-tested than the `indian-stock-market` package that failed earlier, but still liable to break when NSE changes its endpoints

Two cautions worth writing into the report:

1. **Broker APIs carry order-placement capability.** Wiring live trading credentials into a project shared by 7 people is a real liability if a key leaks. If a broker API is used, restrict it to data endpoints, keep keys out of version control, and note this in the threat model — it is a legitimate finding for the cyber half of the project.
2. **SEBI has tightened rules on retail algo trading**, with registration and approval requirements now part of the picture. Purely offline research on historical data is unaffected, but anything that places live automated orders is not a casual student exercise. Keep the project on the research/simulation side of that line.

8. Honest summary for the report

A gradient-boosted model was trained on 4,156 hourly EUR/USD bars using 17 stationary technical features, including a custom ATR trailing-stop indicator ported from Pine Script. Under strict chronological splitting, held-out test accuracy was 54.8% against a 52.7% majority-class baseline (ROC-AUC 0.553). Walk-forward validation across five folds gave a mean accuracy of 50.5% ($\sigma = 2.6\%$) and a mean edge of -1.8% , indicating the apparent test-set edge is within noise. This is consistent with the efficient-market expectation for next-bar direction on a highly liquid pair, and is reported as a finding rather than presented as predictive performance. The pipeline, leakage controls, and evaluation methodology are the substantive contributions; improving on the result requires a better-posed target (triple-barrier or volatility-scaled labels), substantially more history, and cost-aware backtesting rather than a larger model.

A project that reports a null result with correct methodology is stronger than one reporting 95% accuracy from a shuffled split. The second is a bug; the first is science.